Capítulo 1

Análisis

1.1. Introducción

En esta sección se analizarán los requisitos funcionales del sistema y se definirá el alcance funcional del proyecto.

Para facilitar su desarrollo y comprensión, el trabajo se ha estructurado en varios bloques que pueden abordarse de forma modular:

- Diseño del IDE.
- Diseño del sistema de plugins.
- Desarrollo de un juego de ejemplo.
- Desarrollo de un manual de uso.

El desarrollo de estos bloques se realizará de manera incremental, ya que cada uno de ellos depende en cierta medida de la implementación del anterior. Esta aproximación permite validar progresivamente el funcionamiento del sistema a medida que se incorporan nuevas funcionalidades.

Asimismo, se definirán los distintos casos de uso del sistema, en los que el único actor será el usuario, que será el único que interactúa con la herramienta.

1.2. Objetivos y funciones

El objetivo de este proyecto es el desarrollo de una herramienta multiplataforma que permita la creación de software en lenguaje C para sistemas de 8 bits. Asimismo, la herramienta deberá ser lo suficientemente versátil como para permitir la incorporación de nuevas funcionalidades mediante un sistema de plugins.

El sistema deberá integrar inicialmente las librerías de GBDK-2020 y Z88DK, junto con sus respectivos compiladores basados en el compilador SDCC. Gracias a esta integración, la herramienta podrá compilar software para más de 150 plataformas diferentes [**?**] [**?**], la mayoría de ellas basadas en el procesador Z80 de Zilog.

Entre las funciones principales que deberá implementar el sistema se encuentran las siguientes:

- Funcionamiento en sistemas Windows 10.
- Posibilidad de compilar el código fuente en sistemas Linux y macOS.
- Integración con los entornos GBDK-2020 y Z88DK.
- Capacidad para compilar software para múltiples plataformas de 8 bits.
- Visualización de errores y advertencias generados durante la compilación.
- Inclusión de resaltado de sintaxis para el lenguaje C.
- Gestión básica de proyectos y archivos fuente.
- Integración con emuladores para la ejecución del software generado.
- Arquitectura extensible basada en plugins.
- Inclusión de ejemplos de proyectos de referencia.
- Incorporación de un manual de uso para el usuario.

1.3. Requisitos mínimos y restricciones

Para el correcto funcionamiento de la aplicación, se deben cumplir una serie de requisitos mínimos en el equipo en el que se instale:

- Java 17 o superior
- 4 GB de memoria RAM como mínimo (8 GB recomendados)
- 200 MB de espacio disponible en disco
- Compilador de C (como Z88DK, GBDK, entre otros)

En cuanto a las restricciones, no se contemplan limitaciones significativas en su uso. No obstante, al tratarse de una aplicación desarrollada en Java, aunque es multiplataforma, no está diseñada para su ejecución en dispositivos móviles ni tabletas.

1.4. Modelo de desarrollo utilizado

Para el desarrollo de este proyecto se ha seguido un enfoque iterativo o incremental. A partir de un primer prototipo muy básico, se han ido añadiendo nuevas funcionalidades y probándolas de forma progresiva.

En proyectos de cierta complejidad, este enfoque presenta la ventaja de permitir la detección y corrección temprana de errores, antes de su integración en el conjunto principal del sistema.

Este método ha facilitado la adaptación a posibles cambios en los requisitos, así como la mejora continua de la aplicación. No obstante, se ha identificado como principal limitación la falta de una planificación más estructurada, lo que ha derivado en una menor organización en las fases iniciales del proyecto.

1.5. Tecnologías

1.5.1. Introducción

Este proyecto, es fundamentalmente una aplicación desarrollada en Java, lo que condiciona en gran medida las tecnologías empleadas en su desarrollo. Como lenguaje de programación se ha utilizado Java, mientras que para el desarrollo diario se ha empleado el entorno de desarrollo integrado IntelliJ IDEA.

Para la interfaz gráfica se ha optado por el uso de la librería JavaFX, que, aunque el ecosistema Java no destaca especialmente por el desarrollo de interfaces gráficas modernas, permite dotar a la aplicación de un aspecto más actual y visualmente atractivo.

El control de versiones y la gestión del proyecto se han llevado a cabo mediante el uso conjunto de Maven y GitHub.

1.5.2. Java

Java es un lenguaje de programación de propósito general, orientado a objetos y concurrente, diseñado para facilitar el desarrollo de aplicaciones de manera sencilla y segura. Está basado en clases y presenta similitudes con lenguajes como C y C++, aunque simplifica su uso al eliminar ciertas características complejas e incorporar mecanismos que mejoran la seguridad y la portabilidad. Se trata de un lenguaje de tipado estático, en el que los errores pueden detectarse en tiempo de compilación, diferenciándose claramente de los errores en tiempo de ejecución. Además, Java se caracteriza por su independencia de plataforma gracias al uso de la máquina virtual Java (JVM), que permite ejecutar el mismo código en distintos sistemas. Asimismo, incluye gestión automática de memoria mediante un recolector de basura, lo que reduce los problemas asociados a la gestión manual de memoria y contribuye a la robustez del software.[**?**]

Originalmente, Java fue anunciado en mayo de 1995 por Sun Microsystems, aunque no fue lanzado al mercado hasta el 23 de enero de 1996 [**?**]. En la actualidad, Java pertenece a la empresa Oracle Corporation, tras la adquisición de Sun Microsystems por parte de esta.

La elección de Java como lenguaje principal del proyecto se fundamenta en varios aspectos clave. En primer lugar, su carácter multiplataforma permite desarrollar aplicaciones que pueden ejecutarse en distintos sistemas operativos sin necesidad de modificaciones, lo que aporta versatilidad y facilita su integración en diferentes entornos de trabajo. En segundo lugar, Java dispone de un amplio ecosistema de herramientas y tecnologías, como JavaFX o Maven, que resultan fundamentales para el desarrollo de interfaces gráficas y la gestión de dependencias del proyecto. Por último, su enfoque basado en la Programación Orientada a Objetos (POO) favorece la escalabilidad y el mantenimiento del software, lo que contribuye a una mejor organización del sistema.

1.5.3. Maven

Maven es una herramienta para la gestión y construcción de proyectos software que va más allá de la simple compilación del código. A través de la gestión del archivo pom.xml (Project Object Model), proporciona un modelo de construcción uniforme y estructurado. Además, se basa en el principio de "convención sobre configuración", lo que simplifica la organización del proyecto y reduce la necesidad de configuraciones explícitas.

Maven incorpora objetivos (goals) predefinidos que permiten ejecutar tareas específicas, como la compilación, las pruebas o el empaquetado del software. En este proyecto, se ha utilizado para la gestión de dependencias y para automatizar las distintas fases del ciclo de construcción, incluyendo la compilación y el empaquetado de la aplicación.

1.5.4. GitHub

GitHub es una plataforma web que ofrece servicios de alojamiento de código fuente y facilita la colaboración entre desarrolladores. Utiliza Git como sistema de control de versiones y permite el trabajo colaborativo en equipo. En la actualidad, GitHub pertenece a Microsoft y dispone de numerosas funcionalidades orientadas a la colaboración, lo que facilita tanto el mantenimiento de proyectos open source como la participación en ellos.

Por su parte, Git es un sistema de control de versiones distribuido desarrollado por Linus Torvalds en el año 2005, y se ha convertido en el estándar de facto en la industria dentro de su ámbito.

En este proyecto, se ha utilizado GitHub como plataforma de alojamiento, estando disponible el repositorio en la siguiente dirección: `https://github.com/ginjol83/samaruc-editor`. Esto no solo garantiza la trazabilidad del desarrollo, sino que también facilita su acceso, revisión y posible colaboración por parte de otros desarrolladores.

1.5.5. JavaFX

JavaFX es una librería estándar de Java, incorporada desde versiones posteriores, que se utiliza para el desarrollo de interfaces gráficas modernas y atractivas. Está basada en componentes (botones, ventanas, layouts, entre otros) y permite la separación entre la lógica y la presentación mediante el uso de archivos FXML.

JavaFX se diseñó como un relevo natural de Java Swing, que había quedado desactualizado en comparación con otras soluciones del mercado. Además de permitir el uso de archivos de presentación basados en XML, incorpora el uso de hojas de estilo CSS para la personalización de la interfaz.

En el desarrollo de este proyecto, JavaFX ha permitido una correcta separación de la capa de presentación dentro del marco del patrón MVC (Model-View-Controller), además de contribuir a la escalabilidad del proyecto y aportar una imagen visual más atractiva.

1.5.6. IntelliJ IDEA

IntelliJ IDEA es un entorno de desarrollo integrado (IDE) desarrollado por JetBrains, orientado principalmente al desarrollo de software en lenguajes como Java y Kotlin, aunque también puede utilizarse como un editor multipropósito para otros lenguajes de programación.

Según su documentación oficial [**?**], IntelliJ IDEA es un IDE diseñado para el desarrollo profesional, cuyo objetivo es maximizar la productividad del desarrollador. Para ello, incorpora funcionalidades como autocompletado inteligente de código, análisis estático y herramientas avanzadas de refactorización, automatizando tareas rutinarias y repetitivas.

CHAPTER 1 GETTING STARTED

Table 1-1. *IDEA Community vs. Ultimate Editions*

		Ultimate edition	Community edition
Language support	Java	✓	✓
	Groovy	✓	✓
	Kotlin	✓	✓
	Scala (via plugin)	✓	✓
	Python and Jython (via plugin)	✓	✓
	Dart (via plugin)	✓	✓
	Rust (via plugin)	✓	✓
	HTML, XML, JSON, YAML	✓	✓
	XSL, XPath	✓	✓
	Markdown	✓	✓
	JavaScript, TypeScript	✓	
	CoffeeScript, ActionScript	✓	
	SQL	✓	
	CSS, Saas, SCSS, Less, Stylus	✓	
	Ruby and JRuby	✓	
	PHP	✓	
	Go	✓	

(continued)

Figura 1.1: Lenguajes soportados por IntelliJ IDEA según el libro de Hagos (Beginning IntelliJ IDEA)[**?**].

La elección de este entorno para el desarrollo del presente proyecto se justifica por su amplia adopción en el ámbito profesional del desarrollo en Java, así como por la gran cantidad de documentación disponible, tanto por parte de JetBrains como de la comunidad. Además, ofrece un alto grado de personalización, lo que mejora la experiencia de desarrollo y facilita la adaptación a las necesidades del programador. En este IDE se ha desarrollado íntegramente el código fuente en Java, así como la gestión de dependencias y la construcción del proyecto mediante Maven.

1.5.7. Emulicious

Emulicious es un emulador multiplataforma que permite la ejecución de ROMs de diversas plataformas de 8 bits, aunque su sistema principal objetivo es la Game Boy. Fue desarrollado por Michael Stegmaier en 2013 y, desde entonces, ha recibido actualizaciones periódicas. Además, es un proyecto de código abierto (open source) y cuenta con una comunidad de desarrollo muy activa, con su propio canal en Discord. Está desarrollado íntegramente en Java, lo cual se aprovecha en este proyecto, ya que permite su integración directa en el editor como una biblioteca Java adicional.

Se trata de un emulador con funcionalidades avanzadas orientadas al desarrollo, entre las que destaca la inclusión de un depurador de código ensamblador que permite seguir la evolución de los registros y facilitar la depuración de funcionalidades. Entre sus características se encuentran el resaltado de sintaxis personalizable, el registro de trazas, los puntos de interrupción y los puntos de observación. Además, permite cargar archivos fuente para su visualización directa, evitando así la necesidad de desensamblar la ROM [**?**].

El hecho de incorporar este conjunto de herramientas específicas para el desarrollo de software hace que se trate de un emulador claramente orientado al desarrollador, en contraposición a otros más enfocados al usuario final, como se puede observar en la siguiente captura.

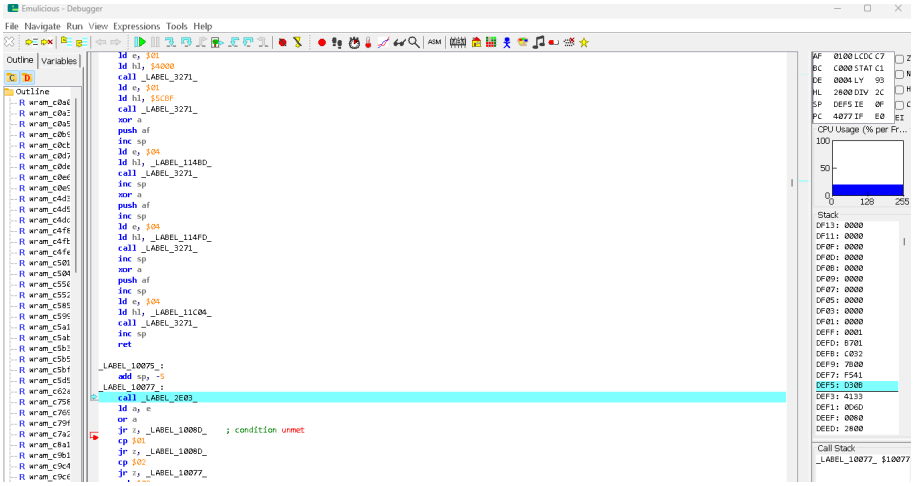

Figura 1.2: Interfaz del debugger integrado de Emulicious mostrando las herramientas de desarrollo disponibles.

El listado de plataformas soportadas es el siguiente:

- Nintendo Game Boy
- Nintendo Game Boy Color
- Sega Master System
- Sega Game Gear
- MSX

1.5.8. JSpeccy

JSpeccy es un emulador de ZX Spectrum desarrollado en Java por José Luis Sánchez [?]. Su objetivo principal es reproducir de forma fiel el comportamiento del hardware original, permitiendo la ejecución de software diseñado para este sistema.

Al igual que otras herramientas utilizadas en este proyecto, JSpeccy está desarrollado íntegramente en Java, lo que facilita su integración dentro del entorno del editor como una biblioteca adicional. Esta característica permite ejecutar y probar directamente los programas desarrollados sin necesidad de depender de aplicaciones externas, simplificando el flujo de trabajo.

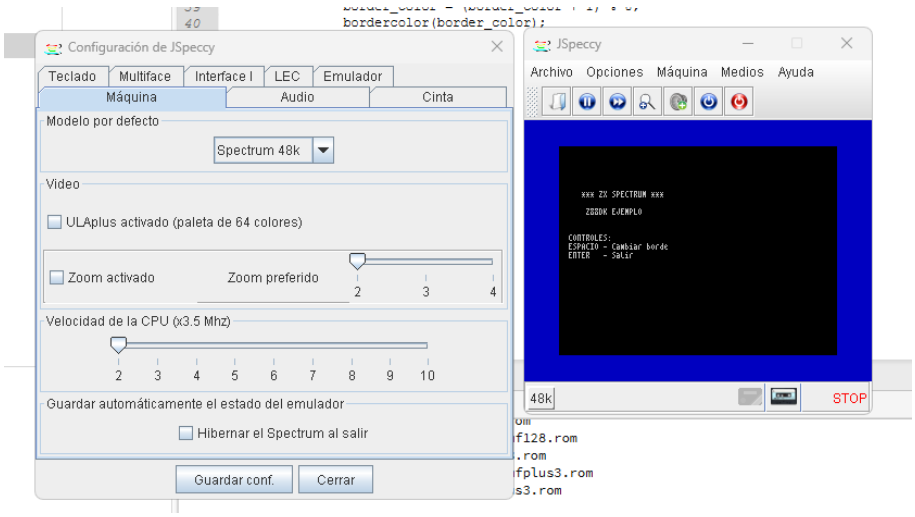

Figura 1.3: Interfaz de JSpeccy y sus diferentes opciones de configuración.

El emulador ofrece soporte para distintos modelos de ZX Spectrum y es compatible con formatos habituales como instantáneas (SNA, Z80) y cintas (TAP, TZX), lo que permite cargar y ejecutar tanto programas como juegos de forma sencilla. Además, incluye soporte para periféricos como el joystick Kempston, ampliando las posibilidades de interacción con el software emulado.

JSpeccy está concebido principalmente como una herramienta de ejecución y validación del software desarrollado, más que como un entorno avanzado de depuración. No obstante, su fidelidad en la emulación y su integración en Java lo convierten en una opción adecuada dentro del flujo de desarrollo planteado en este proyecto.

1.6. Descripción del modelo

En esta sección se describen los requisitos del sistema, entendidos como el conjunto de funcionalidades y restricciones que debe cumplir el entorno de desarrollo propuesto.

Dichos requisitos se presentan desde un punto de vista externo, centrado en el usuario, sin entrar en detalles de implementación interna.

Se distinguen dos tipos de requisitos:

- Requisitos funcionales (RF), que describen las capacidades y servicios que ofrece el sistema.
- Requisitos no funcionales (RNF), que definen atributos de calidad y restricciones del sistema, como el rendimiento, la usabilidad o la portabilidad.

1.6.1. Requisitos

A continuación, se presenta el listado de requisitos identificados para el sistema.

1.6.1.1. Listado de requisitos

Requisitos funcionales

- RF-01: El sistema deberá permitir la creación de nuevos proyectos.
- RF-02: El sistema deberá permitir abrir proyectos existentes.
- RF-03: El sistema deberá permitir guardar proyectos.
- RF-04: El sistema deberá permitir la creación, edición y eliminación de archivos de código.
- RF-05: El sistema deberá permitir compilar proyectos utilizando diferentes toolchains (GBDK, z88dk).
- RF-06: El sistema deberá mostrar los errores de compilación al usuario.
- RF-07: El sistema deberá permitir ejecutar el programa compilado en un emulador.
- RF-08: El sistema deberá permitir la configuración de rutas de compiladores y emuladores.
- RF-09: El sistema deberá soportar la instalación de plugins.
- RF-10: El sistema deberá permitir la gestión (activar/desactivar) de plugins.

Requisitos no funcionales

- RNF-01: El sistema deberá ser multiplataforma (Windows principalmente).
- RNF-02: La interfaz deberá ser intuitiva y fácil de usar.
- RNF-03: El sistema deberá responder a las acciones del usuario en menos de 1 segundo en condiciones normales.
- RNF-04: El sistema deberá permitir la extensión mediante plugins sin modificar el núcleo.
- RNF-05: El sistema deberá estar desarrollado en Java.

1.6.2. Casos de uso

Los casos de uso describen las interacciones entre los actores y el sistema, representando de forma clara cómo se utilizan las funcionalidades definidas en los requisitos. Permiten modelar el comportamiento del sistema desde el punto de vista del usuario, mostrando los distintos escenarios de uso.

En este proyecto, el actor principal es el usuario desarrollador, que utiliza el entorno para crear, editar, compilar y ejecutar programas para sistemas de 8 bits.

A continuación, se describen los principales casos de uso identificados:

- **CU-01: Crear proyecto** - El usuario crea un nuevo proyecto especificando su nombre y ubicación en el sistema de archivos.
- **CU-02: Abrir proyecto** - El usuario selecciona un proyecto existente para cargarlo en el entorno de desarrollo.
- **CU-03: Guardar proyecto** - El usuario guarda los cambios realizados en el proyecto actual.
- **CU-04: Gestionar archivos de código** - El usuario puede crear, editar y eliminar archivos de código fuente dentro del proyecto.
- **CU-05: Compilar proyecto** - El usuario ejecuta el proceso de compilación utilizando una toolchain seleccionada (GBDK o z88dk).
- **CU-06: Visualizar errores de compilación** - El sistema muestra los errores generados durante la compilación, permitiendo al usuario corregirlos.
- **CU-07: Ejecutar programa en emulador** - El usuario ejecuta el programa compilado en un emulador integrado o externo.
- **CU-08: Configurar entorno** - El usuario define las rutas de compiladores y emuladores necesarios para el funcionamiento del sistema.
- **CU-09: Gestionar plugins** - El usuario instala, activa o desactiva plugins para extender la funcionalidad del entorno.

1.6.2.1. Diagrama de casos de uso

Los diagramas de casos de uso permiten representar de manera visual las interacciones entre los actores y el sistema, facilitando la comprensión de los distintos escenarios de uso. A través de estos diagramas, es posible observar de un vistazo qué funcionalidades ofrece el sistema y cómo se relacionan con los usuarios.

En este proyecto, el actor principal es el usuario desarrollador, que realiza acciones sobre el entorno de desarrollo para crear, gestionar y ejecutar proyectos de software.

El diagrama principal de casos de uso incluye los siguientes elementos:

- **Actor:** Usuario desarrollador
- **Casos de uso principales:**
  - Crear proyecto
  - Abrir proyecto
  - Guardar proyecto
  - Gestionar archivos de código
  - Compilar proyecto
  - Visualizar errores de compilación
  - Ejecutar programa en emulador
  - Configurar entorno
  - Gestionar plugins

A continuación se presenta el diagrama UML correspondiente, donde se representan los casos de uso y su relación con el actor:

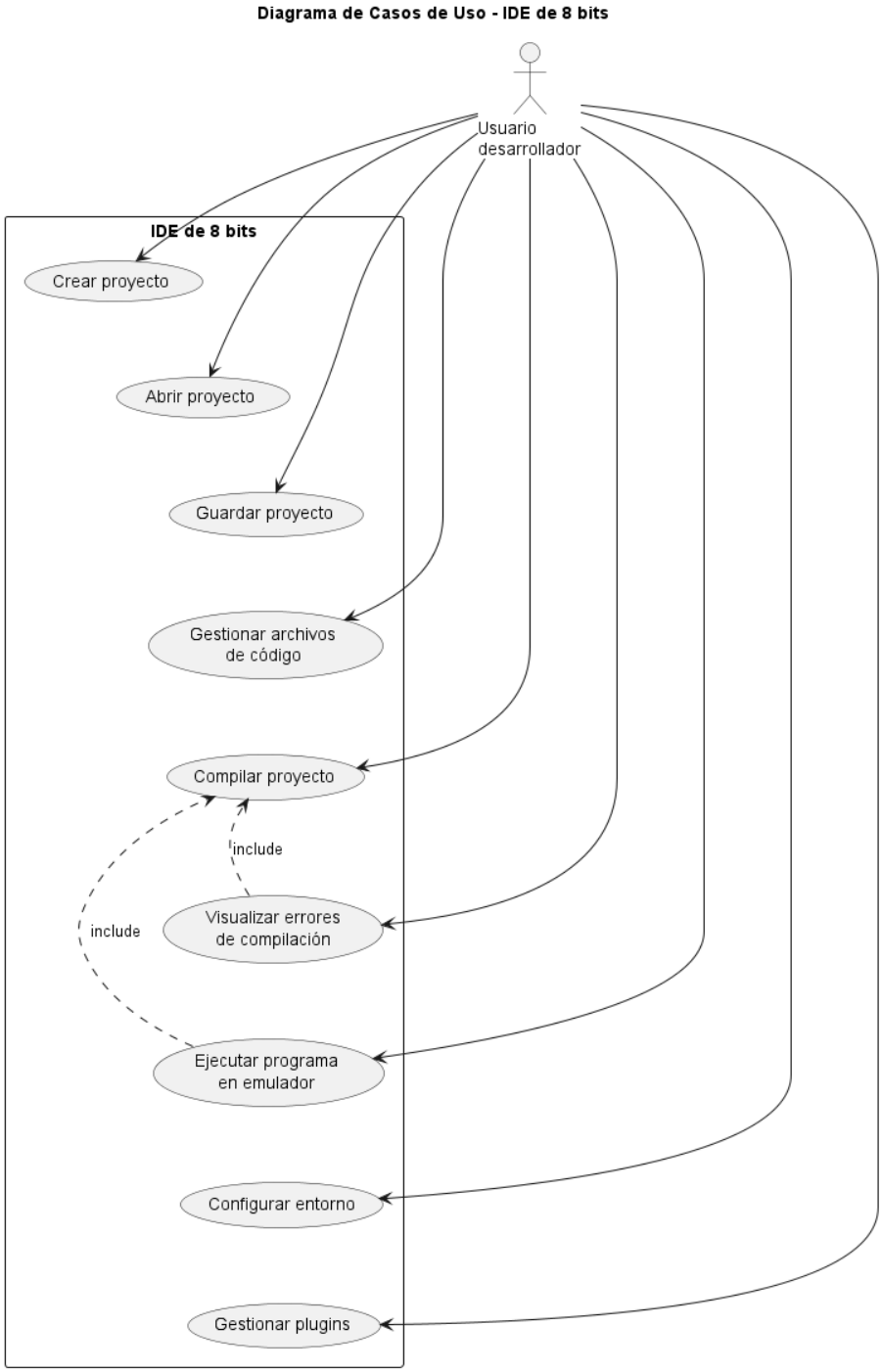

Figura 1.4: Diagrama de casos de uso del sistema de desarrollo IDE.

1.6.3. Casos de uso

Los casos de uso permiten modelar las interacciones entre los actores y el sistema, mostrando cómo se utilizan las funcionalidades definidas en los requisitos. Cada caso de uso describe un escenario específico en el que el usuario interactúa con el sistema, desde el inicio hasta el resultado esperado.

El actor principal es el **usuario desarrollador**, que utiliza el entorno para crear, editar, compilar y ejecutar programas para sistemas de 8 bits. A continuación se describen los principales casos de uso identificados:

CU-01: Crear proyecto

- **Actor:** Usuario desarrollador
- **Precondición:** Ningún proyecto abierto o el usuario desea iniciar un nuevo proyecto
- **Flujo principal:**
  1. El usuario selecciona la opción "Nuevo proyecto".
  2. El sistema solicita nombre y ubicación del proyecto.
  3. El usuario introduce los datos y confirma.
  4. El sistema crea el proyecto y lo abre en el entorno.
- **Postcondición:** Proyecto creado y listo para añadir archivos de código.

CU-02: Abrir proyecto

- **Actor:** Usuario desarrollador
- **Precondición:** Existen proyectos previamente guardados
- **Flujo principal:**
  1. El usuario selecciona "Abrir proyecto".
  2. El sistema muestra un selector de proyectos disponibles.
  3. El usuario selecciona un proyecto y confirma.
  4. El sistema carga el proyecto en el entorno.
- **Postcondición:** Proyecto cargado y listo para edición o compilación.

CU-03: Guardar proyecto

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. El usuario selecciona "Guardar proyecto".
  2. El sistema guarda todos los archivos y configuraciones del proyecto.
- **Postcondición:** Cambios del proyecto guardados de manera segura.

CU-04: Gestionar archivos de código

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. El usuario crea, edita o elimina archivos de código dentro del proyecto.
  2. El sistema refleja los cambios en la estructura del proyecto.
- **Postcondición:** Archivos actualizados correctamente.

CU-05: Compilar proyecto

- **Actor:** Usuario desarrollador
- **Precondición:** El proyecto contiene archivos de código válidos
- **Flujo principal:**
  1. El usuario selecciona la opción "Compilar".
  2. El sistema utiliza la toolchain configurada para generar el ejecutable.
  3. El sistema muestra los resultados de la compilación (errores o éxito).
- **Postcondición:** Archivo ejecutable generado si no existen errores.

CU-06: Visualizar errores de compilación

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. Tras la compilación, el sistema muestra los errores encontrados.
  2. El usuario analiza los errores y realiza correcciones en el código.
- **Postcondición:** El usuario dispone de la información necesaria para corregir errores.

CU-07: Ejecutar programa en emulador

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. El usuario selecciona "Ejecutar en emulador".
  2. El sistema ejecuta el programa compilado en Emulicious o JSpeccy según la configuración.
- **Postcondición:** Programa ejecutado y comportamiento observable por el usuario.

CU-08: Configurar entorno

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. El usuario accede a la configuración del sistema.
  2. Define rutas de compiladores y emuladores.
- **Postcondición:** Sistema preparado para compilar y ejecutar programas correctamente.

CU-09: Gestionar plugins

- **Actor:** Usuario desarrollador
- **Flujo principal:**
  1. El usuario instala, activa o desactiva plugins.
  2. El sistema aplica los cambios de manera inmediata.
- **Postcondición:** Plugins activos o desactivados según la decisión del usuario.

1.6.4. Diagrama de estados

El diagrama de estados describe los distintos estados por los que puede pasar el sistema durante su ejecución, así como las transiciones entre dichos estados en función de las acciones del usuario o de eventos internos.

En este caso, se modela el comportamiento del entorno de desarrollo en relación con la gestión de proyectos, desde su creación o apertura hasta su compilación y ejecución.

El sistema comienza en un estado inicial en el que no hay ningún proyecto cargado. A partir de ahí, el usuario puede crear o abrir un proyecto, pasando al estado de proyecto cargado. Desde este estado, el usuario puede editar el código, compilar el proyecto o cerrar el mismo.

Durante el proceso de compilación, el sistema puede transitar a un estado de error de compilación si se detectan fallos, o a un estado de compilación correcta en caso de éxito. En este último caso, el usuario puede ejecutar el programa en un emulador.

Finalmente, el sistema puede volver a estados anteriores en función de las acciones del usuario, como la corrección de errores o el cierre del proyecto.

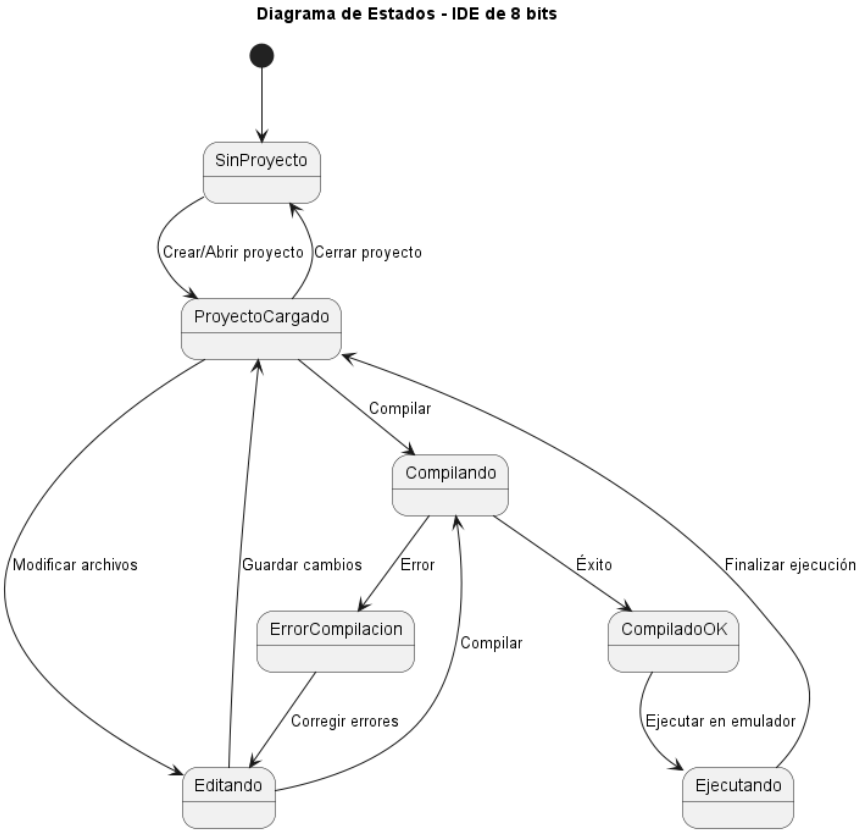

Figura 1.5: Diagrama de estados del entorno de desarrollo.

1.6.5. Diagrama de clases

El diagrama de clases representa la estructura estática del sistema, mostrando las clases principales, sus atributos, métodos y las relaciones entre ellas.

En este proyecto, se ha diseñado un modelo orientado a objetos que refleja los distintos componentes del entorno de desarrollo. La clase principal es el IDE, que actúa como núcleo del sistema y coordina las operaciones principales, como la gestión de proyectos, la compilación y la ejecución.

El sistema se organiza en torno a la clase Proyecto, que contiene los archivos de código fuente. Para la compilación y ejecución, se utilizan las clases Compilador y Emulador, respectivamente. Además, se incluye un sistema de extensión mediante plugins, gestionado por la clase GestorPlugins.

Por último, la clase Configuracion permite almacenar los parámetros necesarios para el funcionamiento del sistema, como las rutas de compiladores y emuladores.

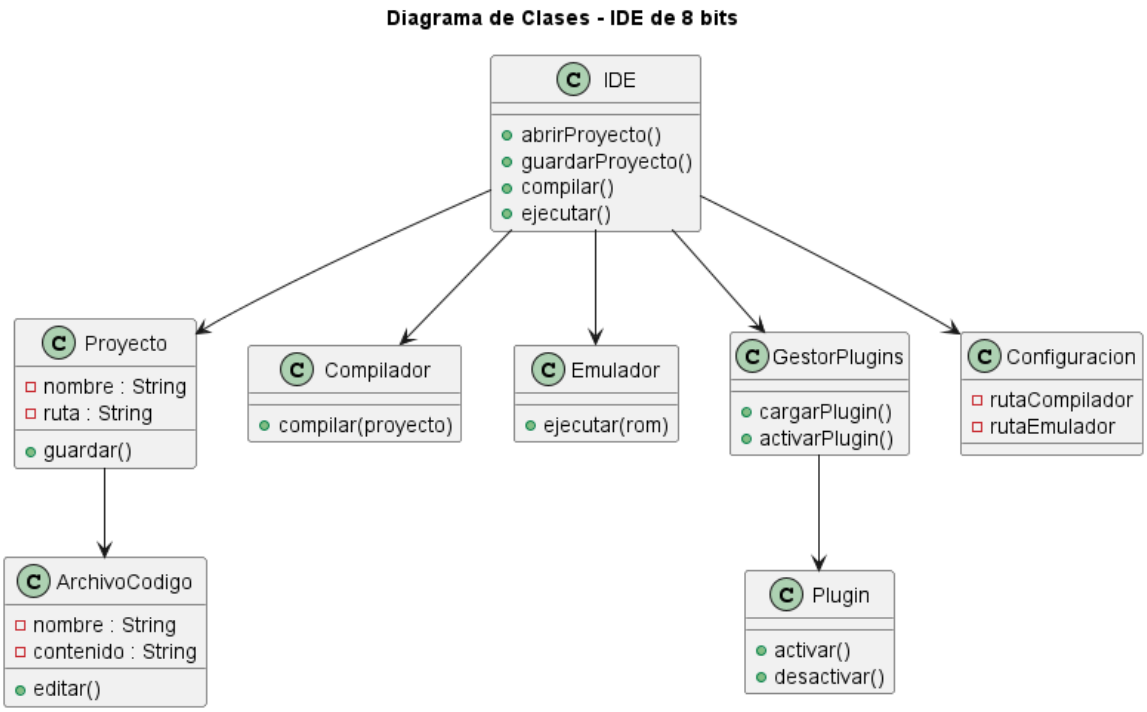

Figura 1.6: Diagrama de clases del sistema.